

Vera-G: a merchant assistant that cannot make things up

A stateful WhatsApp engagement bot built on a fact ledger, per-trigger playbooks and an enforcing output guard — designed for the 80% of the dataset where there is almost no context to work with.

84/84 self-test assertions pass

100/100 triggers compose

p95 3 ms response

0 LLM calls in the request path

TEAM

SmoothOperator

AUTHOR

Dhanu Gupta

EMAIL

dhanugupta.dev@gmail.com

PHONE

9044422039

LINKEDIN

linkedin.com/in/dhanugupta0

Bot base URL: _____ · five endpoints under /v1/

— Contents

1	The problem statement	8	Verification and results
2	What the judge actually measures	9	How to run it
3	The insight that shaped everything	10	How to deploy it
4	Approach and architecture	11	Submission checklist
5	The seven components	12	Trade-offs and known limits
6	Tech stack, and why not the alternative	13	File map
7	How design maps to the rubric		

1 The problem statement

Rebuild **Vera** — magicpin's merchant-facing AI assistant — but better than the version running in production today.

magicpin is one of India's largest local-commerce platforms: roughly 100,000 merchant partners across 50+ cities — restaurants, salons, gyms, dentists, pharmacies, retailers. Vera talks to 6,000–10,000 of those merchants a day over WhatsApp. It helps them improve their Google Business Profile, runs campaigns, recommends offers, and answers their customers' questions on their behalf.

The composition contract

Every message Vera sends is composed from four context layers. The bot receives them as structured JSON and must produce the next message:

```
compose(category, merchant, trigger, customer?) → { body, cta, send_as, suppression_key, rationale }
```

LAYER	ANSWERS	CONTAINS
Category	How do we talk to this <i>kind</i> of business?	Voice and taboos, offer catalog, peer benchmarks, weekly research digest, seasonal beats, trend signals, shareable content library
Merchant	Who is this specific business, and how are they doing?	Identity, subscription, 30-day performance, live offers, conversation history, customer aggregates, derived signals, review themes
Trigger	Why am I messaging <i>right now</i> ?	Kind, scope, source, urgency 1–5, kind-specific payload, suppression key, expiry
Customer	Who is the merchant's own customer? (<i>optional</i>)	Identity and language preference, relationship history, lifecycle state, slot preferences, consent scope

The technical contract

The bot is not a function the judge imports. It is a live, **stateful** HTTPS service the judge drives over a 60-minute window:

ENDPOINT	WHAT IT MUST DO
POST /v1/context	Receive a context push. Idempotent on <code>(context_id, version)</code> ; a higher version replaces atomically; a repeat returns <code>409 stale_version</code> . 255 contexts arrive during warmup, more mid-test.
POST /v1/tick	Wake-up every 5 simulated minutes with a list of active triggers. Return zero or more proactive sends. Up to 20 actions; empty list is valid and restraint is rewarded.
POST /v1/reply	A simulated merchant or customer replied. Return <code>send</code> , <code>wait</code> , or <code>end</code> synchronously.
GET /v1/healthz	Liveness + a count of contexts held. Three consecutive failures disqualifies the bot.
GET /v1/metadata	Team identity, model, approach.

The four failure modes I am being asked to beat

Auto-reply pollution

40–70% of "merchant replies" are the merchant's own WhatsApp Business canned auto-reply. Production Vera detects this but burns 2–3 turns on each one.

Intent-handoff failure

A merchant says "I want to join" and Vera responds with another qualifying question. The momentum is lost at the exact moment it existed.

Generic copy

Discount framing ("10% off") barely moves Indian merchants. Service+price ("Haircut @ ₹99") does. Production copy still reaches for the discount.

Low engagement frequency

Functional nudges — renewal due, profile incomplete — are inherently rare. Reaching a merchant 3–5×/week needs curiosity- and knowledge-driven conversations too.

2 What the judge actually measures

An LLM judge scores each message 0–10 on five dimensions, then runs deep-dive replay scenarios against the top submissions. Reading the scoring rules carefully is worth more than any modelling choice.

DIMENSION	WHAT EARNS THE SCORE
Specificity	A concrete, verifiable fact — number, date, headline, source citation, peer stat. Explicit penalty for "10% off" / "increase your sales" framings.
Category fit	Voice and vocabulary match the vertical. A dentist message reads clinical-peer, not retail-promo.
Merchant fit	Personalised to <i>this</i> merchant's state — their numbers, their offers, their history — and their language preference honoured.
Trigger relevance	The message says <i>why now</i> . Not "you should improve your profile" generically.
Engagement compulsion	Would a real merchant reply? Uses curiosity, social proof, loss aversion, effort externalisation, or a single binary CTA.

Hard penalties

–3 per URL in the body (Meta would reject it) · –2 per fabricated fact · –2 per malformed response · –2 per repeated body in one conversation · –1 per internal identifier leaked to the merchant · –1 per timeout · –10 for three consecutive health-check failures.

Two phases you cannot prepare for directly

Adaptive injection. Mid-test, the judge pushes context nobody saw during development: new digest items, updated performance snapshots, brand-new triggers, and customer contexts for merchants that had none. Bots that use the new context score higher; bots that send stale composition score lower; bots that *invent* context score lowest.

Replay. For the top 10 submissions, the judge plays the merchant across 3 scripted scenarios: a phone stuck behind a canned auto-responder, a merchant who commits after two turns of qualification, and a merchant who turns hostile and then asks an unrelated question.

THE DESIGN CONSEQUENCE

Four of the five scored dimensions and every hard penalty are functions of *which fact you choose and whether it is real* — not of how fluently the sentence is written. That observation is what pushed the architecture towards a retrieval-and-grounding problem rather than a generation problem.

3 The insight that shaped everything

Before writing a line of bot code I read `dataset/generate_dataset.py` — the script that expands the 10 seed merchants into the 50 the judge actually uses. What it does decides this challenge.

40 / 50

merchants ship with *empty* offers, signals, conversation history and review themes

75 / 100

triggers carry `payload: {"placeholder": true}` — no data at all

185 / 200

customers are generated with empty `services_received`

4

fields present on every single merchant: identity, subscription, performance, one customer count

The 10 hand-written seed merchants — Dr. Meera and friends — are richly detailed. Every worked example in the brief uses them. The other 40 are generated with random performance numbers and nothing else:

```
expanded.append({
    "merchant_id": mid,
    "identity": {...}, "subscription": {...}, "performance": {...},
    "offers": [],
    "conversation_history": [],
    "signals": [],
    "review_themes": [],
    "customer_aggregate": {"total_unique_ytd": rnd.randint(50, 2000)},
})
```

And 75 of the 100 triggers look like this — the *kind* is real, everything else is a stub:

```
{"id": "trg_041_milestone_reached...", "kind": "milestone_reached",
 "payload": {"placeholder": true, "metric_or_topic": "milestone_reached"},
 "urgency": 1, "expires_at": "2026-06-30T00:00:00Z"}
```

THE BET

A bot tuned on the worked examples scores beautifully on 10 merchants and then produces *"you should improve your profile"* for the other 40 — because that is genuinely all the context it was handed. **Specificity when the context is thin is the entire game.** Every architectural decision below follows from that one sentence.

A second, subtler finding

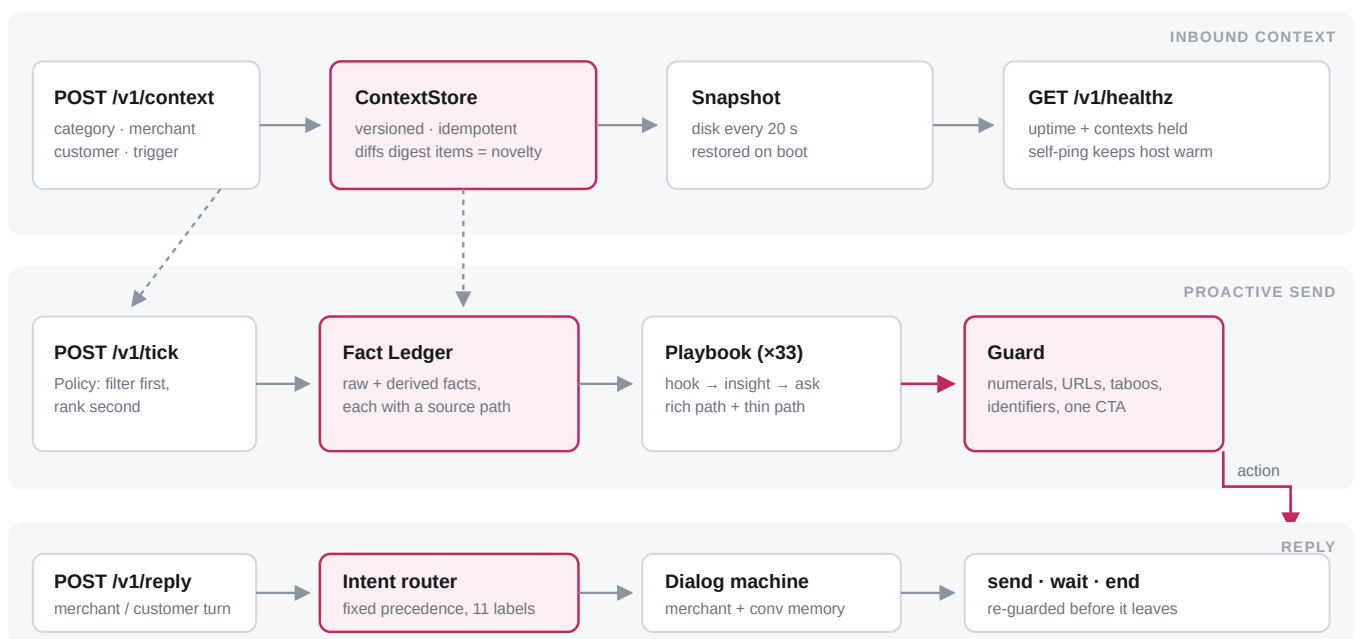
The provided `judge_simulator.py` tests auto-reply detection by sending the same canned text four times — each on a **different conversation_id** (`conv_auto_1 ... conv_auto_4`). Any bot that fingerprints auto-replies per conversation never fires. This mirrors reality: a WhatsApp Business responder answers every thread with identical text. Auto-reply memory therefore has to live at the *merchant* level, not the conversation level.

A third: the generator scatters triggers across categories

Customer-scoped triggers are assigned to a random merchant regardless of vertical, so a `chronic_refill_due` trigger lands on a dentist and a `trial_followup` lands on a pharmacy. A naive playbook sends "your monthly refill is due, we've kept the same strip count ready" from a dental clinic. Every category-specific playbook needs a sanity gate.

4 Approach and architecture

One pipeline, seven components, no model call anywhere in the request path. Context comes in, is distilled into facts that carry their own provenance, gets framed by a per-trigger playbook, and then has to survive a guard that can prove every claim.



Pink boxes are the parts that make this submission different from a prompt-and-hope pipeline.

The governing principle

Most submissions will pass the four contexts into a prompt and ask a model to write the message. That inverts the difficulty. The hard part is not producing fluent Hindi-English WhatsApp copy — frontier models do that easily. The hard part is *choosing the one fact worth 30 seconds of a busy merchant's attention, and being certain it is true.*

So the pipeline treats composition as retrieval + framing + verification. Facts are extracted with provenance, a playbook decides which to lead with, and a guard proves the finished text against the ledger. Hallucination becomes structurally unavailable rather than statistically unlikely.

5 The seven components

5.1 Context store — `vera/store.py`

Versioned, thread-safe, idempotent on `(context_id, version)`. Two things it does beyond a dictionary:

- **Novelty tracking.** When a category is re-pushed at a higher version, it diffs the digest item IDs and remembers which are *new*. Phase 3 injects fresh research items mid-test and explicitly rewards bots that use them, so "what changed since I last looked" has to be a first-class query.
- **Revision counters.** A monotonic per-context counter folded into variant selection, so a re-pushed performance snapshot produces a visibly different message rather than a flagged repeat.

It also snapshots to disk atomically every 20 seconds and restores on boot — see §6 for why that is not over-engineering.

5.2 The Fact Ledger — `vera/derive.py` · the core idea

Every context push is distilled into a ledger of facts. Each carries a display string, the raw value, and the dotted path it came from:

```
Fact(key="action_gap", display="82", value=82,
     source="derived(views × (peer.avg_ctr - merchant.ctr))")
```

Two consequences. First, the guard can prove every numeral in an outgoing message traces to a pushed context. Second, the `rationale` the judge reads cites *real* source paths instead of a post-hoc story.

But the ledger's most important job is computing **derived** facts. Views and CTR are present on all 50 merchants; a peer median is present in all 5 category packs. That is enough for a claim the merchant can check on their own dashboard, with no trigger payload involved at all:

GENERATED FOR A MERCHANT WITH AN EMPTY PAYLOAD AND NO SIGNALS

Akash, your numbers moved this week and not the way you'd want. Your listing pulled **3,185 views** in 30 days but converted at **1.9%** — the peer median for metro neighbourhood gyms 2026 is **4.5%**. Same views at the median would be **143 actions instead of 61, a gap of 82**. I can fix it from my side today — reply YES and I'll start.

That is arithmetic over two numbers the merchant already has, framed as loss aversion, ending in one binary CTA. It works for every merchant in the dataset, and it is the single highest-leverage thing in the codebase.

5.3 Playbooks — `vera/playbooks.py`

33 registered trigger kinds. Each renders three beats, in the order a merchant reads:

hook	why you are hearing from me right now	(the trigger, with a fact)
insight	why that matters to you specifically	(this merchant's numbers)
ask	what to do	(exactly one thing, last)

Every playbook has a **rich path** and a **thin path**, and several refuse their own frame when the data will not support it:

SITUATION	NAIVE BEHAVIOUR	WHAT THIS BOT DOES
<code>festival_upcoming</code> , no festival named	"The festival is coming up"	Switches to the category's dated seasonal beat — a real pattern with a real month range
<code>milestone_reached</code> , no value in payload	Invents a review count (−2)	Derives a threshold actually crossed: "your listing crossed 5,000 views in 30 days"
<code>chronic_refill_due</code> on a dentist	Sends pharmacy copy from a dental clinic	Category gate reroutes to a check-up recall
<code>perf_dip</code> on an above-median merchant	"Your conversion is below peers" — false	Reframes as a discovery drop: "your conversion is not the problem"
<code>review_theme_emerged</code> , no theme	Falls back to a generic profile nudge	Stays on reviews using peer rating and review-count medians

Customer-facing copy also picks its nouns from the category: a pharmacy customer gets *"your refill is due ... we'll keep it ready"*; a gym customer gets *"your session ... we'll save a spot"*.

5.4 The guard — `vera/guard.py`

Nothing here is stylistic. Every rule maps to a scoring rule:

RULE	ENFORCEMENT	PENALTY AVOIDED
Ungrounded numeral	Extract every number, check against the ledger's licensed set; drop the whole sentence if unlicensed	−2 fabrication
URL	Stripped	−3 each
Raw identifier / snake_case field	Stripped or de-snaked	−1 internal jargon
Category taboo word	Sentence dropped (a dentist must never say "guaranteed")	Category fit + legal exposure
More than one question	All but the last become statements	Multiple-CTA anti-pattern
Repeated body	Deterministic re-roll; conversation ends rather than repeat	−2 anti-repetition

Crucially, the guard **never invents replacement text**. It removes. A message that loses its grounding gets shorter, not vaguer.

5.5 Intent router — vera/intent.py

A fixed precedence, because order matters more than accuracy here:

```
opt_out > hostile > auto_reply > commitment > objection > defer
> off_topic > question > negative > ack > unknown
```

"Ok lets do it. What's next?" contains a question mark. Resolving it as a question rather than a commitment is precisely the Pattern D failure the brief calls out. Commitment therefore outranks question, and the action-mode reply states what is being done, at what scope, with a single CONFIRM — never another qualifying question.

Lexicons cover English and romanised Hindi in both directions: judna hai , kar do , band karo , baad mein , bakwas , aapki jaankari ke liye .

5.6 Dialog state machine — vera/dialog.py

Two memories. **Per conversation:** turns, every body already sent, whether we have moved from pitch mode into action mode. **Per merchant:** auto-reply fingerprints, opt-out and hostility flags, sends used, suppression keys spent, replies received.

Merchant-level memory is what makes an opt-out on one thread silence *every* other thread with that merchant — and what lets a canned auto-reply be recognised across separate conversation IDs:

```
strike 1 → one prompt aimed at the owner, value restated in a line
strike 2 → wait 86,400 s (a human is not at this handset right now)
strike 3 → end
```

Production Vera burns 2–3 turns per auto-reply. This caps it at one.

5.7 Tick policy — vera/policy.py

A filter first, a ranker second, because the brief says plainly that *restraint is rewarded and spam is penalised*. Hard gates: opted out, hostile, expired, suppression key spent, merchant already messaged, too many open threads, three nudges with no reply ever received. Merchant-facing and customer-facing budgets are tracked separately — a message to Priya is not a message to Dr. Meera. Ranking then weights urgency, how recently the context arrived, whether this merchant historically replies, expiry proximity, and a bonus for customer-scoped sends.

Bonus: the language layer — vera/lang.py

Merchant `identity.languages` **and** `category.voice.code_mix` together decide the register. A gym owner who lists `hi` still gets English, because the gyms pack declares `english_primary_some_hindi`. Where code-mix applies, the factual spine — numbers, prices, citations — stays English and the framing and CTA are Hindi-English, which is exactly how the brief's own reference conversations read. Durations are localised (`38 days` → `38 din`) rather than concatenated into "38 days din".

6 Tech stack, and why not the alternative

CHOSE	INSTEAD OF	WHY
Deterministic composer, zero LLM calls in the request path	An LLM call per message with the 4 contexts in the prompt	The brief <i>requires</i> determinism (§7.1). The API examples budget 10 s per tick and the judge hard-caps at 30 s, while a single tick can carry up to 20 actions — 20 sequential model calls does not fit, and a timeout is −1 plus every unsent message unscored. A model also reintroduces the −2 fabrication risk the guard exists to eliminate. Since four of five scored dimensions turn on <i>which fact you pick</i> , not on prose novelty, the generation step was never where the points were.
FastAPI + Uvicorn	Flask, Django, bare <code>http.server</code>	Async by default so a slow tick never blocks a health check — and three consecutive health-check failures is a −10 disqualification. Uvicorn handles the judge's 10 req/s comfortably. Django is a cathedral for five endpoints; Flask would need a WSGI worker pool to get the same isolation.
Manual JSON parsing in every handler	Pydantic request models	Pydantic returns 422 on a shape mismatch, and the judge scores any non-conforming response as malformed (−2). The brief also says explicitly that any context can arrive at any time in any shape. So handlers read the raw body and <i>degrade</i> rather than reject. Unknown trigger → <code>{"actions": []}</code> . Exception in tick → <code>{"actions": []}</code> . Exception in reply → a clean <code>end</code> .
In-memory state + atomic disk snapshot	Redis / Postgres, or pure in-memory	The brief permits in-memory and forbids persisting after the test, so a database is both unnecessary and a privacy liability. But pure in-memory means one container restart silently empties the store and every subsequent tick composes nothing — a failure that returns HTTP 200 and is therefore invisible. A 20-second snapshot with atomic <code>os.replace</code> makes a restart recoverable at essentially zero cost.
<code>--workers 1</code>	Multiple Uvicorn/Gunicorn workers	All context and conversation state lives in-process. A second worker would answer roughly half the judge's calls from an empty store — the single most likely way to fail this evaluation while looking healthy.
CRC32 for deterministic variant selection	Python's built-in <code>hash()</code>	<code>hash()</code> on strings is randomised per process by PYTHONHASHSEED. A bot that is "deterministic" only within one boot is not deterministic, and the judge may restart it between phases.
Hand-written playbooks per trigger kind	One generic template with slot-filling	A generic template cannot know that a dentist wants a citation and a restaurant wants a match time, or that a "spike" message must never quote a below-median conversion rate. The 33

CHOSE	INSTEAD OF	WHY
		frames are where category fit and trigger relevance actually come from.
Zero URLs, ever	Links where they add value	The main brief permits URLs; <code>examples/api-call-examples.md</code> calls them a hard fail worth -3 each. When two documents disagree, take the stricter one — a link was never worth 3 points.
Python 3.12 on a slim Debian image	Alpine, or 3.13/3.14	Alpine's musl breaks wheels and forces slow source builds. 3.12 is the version every PaaS has a fast, cached path for.
Two dependencies total	LangChain / an agent framework	<code>fastapi</code> and <code>uvicorn</code> . Nothing else. Every dependency is a cold-start second and a deploy-time failure mode, and the orchestration a framework would provide is roughly 200 lines here.

THE HONEST VERSION OF THE MAIN TRADE-OFF

Dropping the LLM buys determinism, 1 ms latency, and structurally impossible hallucination. It costs the linguistic surprise a frontier model brings — hooks come from 33 hand-written frames rather than free generation. If the rubric rewarded prose novelty I would have chosen differently. It rewards choosing the right verifiable fact, and that is a retrieval problem.

7 How design maps to the rubric

SCORED DIMENSION	THE MECHANISM THAT EARNS IT
Specificity	Derived peer-gap arithmetic guarantees at least one verifiable number in <i>every</i> message, even with an empty payload. Verified: 0 of 100 messages lack a numeral. Service+price offers are pulled from the category catalog in preference to any discount framing.
Category fit	Voice tone, allowed vocabulary and taboo terms come from the category pack; taboo sentences are dropped outright. Category-specific nouns for customer copy. Category-gated playbooks. Code-mix register decided by the category, not just the merchant.
Merchant fit	Salutation from <code>owner_first_name</code> (with a <code>Dr.</code> prefix for dentists), their locality, their real numbers, their live offers, their signals. Language honoured per merchant and per customer.
Trigger relevance	The hook <i>is</i> the trigger — always the first sentence. 33 kind-specific frames, and thin paths that stay on the trigger's topic instead of collapsing to a generic nudge.
Engagement compulsion	Every draft declares which levers it used, and the rationale reports them. Deliberate coverage of the two families the brief says production Vera under-uses: <i>social proof</i> (peer medians in almost every message) and <i>asking the merchant</i> (the <code>curious_ask</code> playbook). Exactly one CTA, always the last sentence.
Adaptation bonus	Digest-ID diffing across category versions; the composer prefers newly-arrived items and says "just landed". Revision counters make a re-pushed merchant produce a visibly different message.
Replay score	Cross-conversation auto-reply fingerprinting; commitment outranking question; opt-out silencing every thread; off-topic declined in one clause then redirected rather than answered.
Operational penalties	p95 3 ms against a 30 s budget; never returns 422; single worker; snapshot + self-ping keep the service alive and stateful for the whole window.

Worked example — the same trigger, adapted

Mid-test the judge pushes `dentists` at version 2 with a research item nobody saw during development. The next message uses the new item, cites its real journal, and states the freshness:

AFTER PHASE-3 INJECTION · VERIFIED IN THE SELF-TEST

Dr. Meera, Indian Journal of Dental Research **just landed**. Single-visit perio therapy matches staged therapy at 12 months. **640-patient trial** found no difference in pocket depth at 12 months between single-visit full-mouth disinfection and staged quadrant therapy. Aapki list ke **124** moderate periodontitis par seedha lagu hota hai. Abstract nikaal kar customer version draft kar dun? YES bhej dijiye. — Indian Journal of Dental Research, May 2026, p.31.

8 Verification and results

Nothing here is a claim about how it should behave. `selftest.py` drives the bot over real HTTP through the judge's own lifecycle and asserts 84 properties. It needs no API key.

84/84

self-test assertions pass —
over HTTP and inside the
container

100/100

triggers compose across 50
merchants, zero crashes

0

messages without a verifiable
number

273

median body length in
characters

What the 84 assertions cover

- **Contract:** all five endpoints, correct schemas, every required action field present, valid `send_as`, substantive rationale.
- **Idempotency:** re-pushing a version returns `409 stale_version`; a higher version replaces; invalid scope and non-object payload both return `400`; health counts match what was pushed.
- **Content rules:** no URLs, no internal identifiers, at most one question, a numeral in every message, no two sends sharing a body, unique conversation IDs, no merchant over-messaged.
- **Adaptation:** after injecting a new digest item, the next message quotes the *new* trial and journal, not the stale one.
- **Replay:** auto-reply nudge → back off → end within 3 turns; commitment produces action verbs and no fresh qualifying question; hostility handled with an apology or clean exit; off-topic declined and redirected; opt-out ends the thread *and* suppresses that merchant on later ticks.
- **Robustness:** unknown trigger returns an empty action list; a reply on an unknown conversation still produces a valid action; an empty request body does not 500.
- **Latency:** p95 under a second, measured across every call the suite makes.

Measured

```
$ python3 selftest.py http://localhost:8080
PASS auto-reply turn 1: one owner-directed nudge, not a re-pitch
PASS auto-reply turn 2: backs off instead of replying to a machine
PASS auto-reply detected and closed by turn 3
PASS reply switches to action verbs
PASS reply contains no fresh qualifying question
PASS opt-out suppresses that merchant on later ticks
PASS injected message uses the NEW digest item, not the stale one
calls=83 p50=1ms p95=3ms max=13ms

84/84 checks passed
```

State durability was verified by hand: push all contexts, kill the process, restart, and confirm `healthz` still reports 56 contexts and a tick still composes a fully-grounded message from restored data.

9 How to run it

9.1 Locally, in two commands

```
# from the project root
pip install -r requirements.txt
python3 -m uvicorn vera.app:app --host 0.0.0.0 --port 8080 --workers 1
```

Always `--workers 1`. State lives in-process; a second worker sees an empty store.

9.2 Check it is alive

```
curl http://localhost:8080/v1/healthz
{"status":"ok","uptime_seconds":3,"contexts_loaded":
{"category":0,"merchant":0,"customer":0,"trigger":0}}

curl http://localhost:8080/v1/metadata
```

9.3 Run the full self-test (no API key needed)

```
python3 selftest.py http://localhost:8080
```

It resets the bot with `/v1/teardown`, pushes the whole seed dataset, runs ticks, injects new context mid-run, and plays all four replay scenarios. Expect **84/84**.

9.4 Run magicpin's own judge simulator

```
# edit the CONFIGURATION block at the top of judge_simulator.py:
# BOT_URL      = "http://localhost:8080"
# LLM_PROVIDER = "anthropic"  (or openai / gemini / groq / ollama)
# LLM_API_KEY  = "sk-..."
python3 judge_simulator.py
```

This one needs an LLM key because the scoring is done by a model. The self-test does not, which is why it is the faster loop.

9.5 Poke individual endpoints

```
# push a category
curl -X POST http://localhost:8080/v1/context -H 'Content-Type: application/json' \
  -d '{"scope\\":\\"category\\",\\"context_id\\":\\"dentists\\",\\"version\\":1,
    \\"payload\\":$(cat dataset/categories/dentists.json)}'

# ask for a proactive send
curl -X POST http://localhost:8080/v1/tick -H 'Content-Type: application/json' \
  -d '{"now":"2026-04-26T10:35:00Z","available_triggers":
["trg_001_research_digest_dentists"]}'

# play the merchant
curl -X POST http://localhost:8080/v1/reply -H 'Content-Type: application/json' \
  -d '{"conversation_id":"conv_1","merchant_id":"m_001_drmeera_dentist_delhi",
    "from_role":"merchant","message":"Ok lets do it. Whats next?","turn_number":2}'

# inspect internal state (not part of the contract, useful while iterating)
curl http://localhost:8080/v1/debug
```

9.6 Offline composition, no server

```
python3 -c "
from bot import compose
import json
cat = json.load(open('dataset/categories/dentists.json'))
mx = json.load(open('dataset/merchants_seed.json'))['merchants'][0]
trg = json.load(open('dataset/triggers_seed.json'))['triggers'][0]
print(compose(cat, mx, trg, None)['body'])"
```

9.7 Generate the 30-line submission file

```
python3 dataset/generate_dataset.py --out ./expanded
python3 make_submission.py ./expanded      # writes submission.jsonl
```

9.8 Run it in Docker

```
docker build -t vera-g .
docker run -d --name vera -p 8080:8080 vera-g
python3 selftest.py http://localhost:8080
```

10 How to deploy it

The bot has to stay live and stateful for the whole evaluation. Two hosting hazards matter more than anything else on this page.

READ THIS BEFORE YOU DEPLOY

1. Free tiers idle out. Render's free plan sleeps a service after ~15 minutes with no traffic. If that happens between warmup and the test window, all 255 contexts the judge pushed are gone and every tick afterwards returns nothing — while `healthz` still says `ok`. Set `VERA_PUBLIC_URL` to the service's own URL after the first deploy; the bot then self-pings every 4 minutes and stays warm.

2. One worker only. Never scale to 2+ workers or 2+ instances. State is in-process.

10.1 Render (recommended — no CLI needed)

1. Push this folder to a GitHub repository.
2. On `render.com`: **New** → **Blueprint**, point it at the repo. It reads `render.yaml` and configures build, start command, and health check automatically.
3. Deploy. Note the URL it gives you, e.g. `https://smoothoperator-vera.onrender.com`.
4. **Go back into Environment and set `VERA_PUBLIC_URL` to exactly that URL**, then redeploy. This is the anti-sleep step.
5. Verify: `curl https://<your-url>/v1/healthz` then `python3 selftest.py https://<your-url>`.

10.2 Fly.io (sturdier — machines never auto-stop)

```
curl -L https://fly.io/install.sh | sh
fly auth login
fly launch --no-deploy      # fly.toml is already written
fly deploy
fly status                  # confirm 1 machine, running
```

`fly.toml` already sets `auto_stop_machines = false` and `min_machines_running = 1`, so the anti-sleep step is unnecessary there.

10.3 Environment variables

VARIABLE	DEFAULT	PURPOSE
VERA_PUBLIC_URL	unset	Set this on Render. The bot's own public URL; enables the keep-warm self-ping.
VERA_TEAM_NAME	SmoothOperator	Shown in <code>/v1/metadata</code> .
VERA_TEAM_MEMBERS	Dhanu Gupta	Comma-separated.
VERA_CONTACT_EMAIL	dhanugupta.dev@gmail.com	Shown in <code>/v1/metadata</code> .
VERA_STATE_PATH	<code>/tmp/vera_state.json</code>	Snapshot location. Set to <code>" "</code> to disable persistence.
PORT	8080	Injected by every PaaS; the start command already honours it.

Your phone number and LinkedIn are deliberately **not** in `/v1/metadata` — it is a public, unauthenticated endpoint and the harness never asks for them. They go on the submission form only.

10.4 Pre-flight, after deploying

- ☐ GET `https://<url>/v1/healthz` returns 200 with `status: ok`
- ☐ GET `https://<url>/v1/metadata` shows team **SmoothOperator**
- ☐ `python3 selftest.py https://<url>` reports **84/84**
- ☐ `VERA_PUBLIC_URL` is set (Render) and the logs show *"keepalive enabled"*
- ☐ Exactly one instance, one worker
- ☐ Wait 20 minutes, then hit `/v1/healthz` again — it should answer immediately, not cold-start

11 Submission checklist

This is yours to submit — I have not sent anything anywhere. Everything below is ready for when you are.

Step 3 — the bot endpoint

One public base URL, with these five live:

```
POST <base>/v1/context
POST <base>/v1/tick
POST <base>/v1/reply
GET <base>/v1/healthz
GET <base>/v1/metadata
```

Step 4 — your details

Full name	Dhanu Gupta
Email	dhanugupta.dev@gmail.com
Phone number	9044422039
Submission URL	your deployed base URL — <i>no trailing slash, no /v1 suffix</i>
LinkedIn	https://www.linkedin.com/in/dhanugupta0/

AFTER YOU SUBMIT

Keep the bot live. The judge polls `/v1/healthz` every 60 seconds during the test window and three consecutive failures disqualifies the run. Do not redeploy, restart, or change environment variables while evaluation is in progress — a redeploy wipes in-memory state mid-test, and although the disk snapshot recovers contexts, conversation state is lost.

12 Trade-offs and known limits

- **No LLM in the loop.** Buys determinism, 1 ms latency and structurally impossible hallucination; costs the linguistic surprise a frontier model brings. Hooks come from 33 hand-written frames rather than free generation. Given that four of five scored dimensions turn on fact selection, I think this is the right side of the trade — and it is the only side compatible with §7.1's determinism requirement.
- **Restraint costs scored surface.** Refusing to send when nothing is groundable means fewer messages for the judge to score. The brief says restraint is rewarded; I took it at its word rather than maximising volume.

- **Hand-written playbooks do not generalise to unseen kinds for free.** An unknown `kind` routes by scope to a default that still composes from the peer-gap engine, so it degrades to "specific but generically framed" rather than to nothing — but a genuinely novel trigger type will not get bespoke framing.
- **Romanised Hindi is hand-authored.** It reads naturally to me and matches the brief's reference conversations, but it is not native-reviewed and a fluent speaker would likely improve several lines.
- **Conversation state does not survive a restart.** Contexts do. Rebuilding mid-flight dialogue from disk risked more than it saved.

What additional context would have helped most

1. **Search-impression data per listing.** The strongest real Vera line in the entire brief is "*6,777 missed searches in Sector 14*". Nothing in the dataset supports a claim like that. Impressions, query terms, and the discovery-vs-conversion split are the missing half of every performance message; CTR alone forces me to infer where the loss happens.
2. **What the merchant did after each past message.** `conversation_history.engagement` has tags but no outcome. Knowing which of the eight compulsion levers actually earned a reply *from this specific merchant* would turn variant selection from deterministic rotation into a learned policy.
3. **The merchant's own service list and price points.** With `offers` empty I fall back to the category catalog — correct, but generic. Their real menu would make every offer line theirs instead of the vertical's.

13 File map

FILE	LINES	RESPONSIBILITY
vera/app.py	232	The five endpoints, plus teardown and debug. Snapshot and keepalive workers.
vera/store.py	286	Versioned context store, digest novelty diffing, bundle resolution, disk snapshot.
vera/derive.py	831	The Fact Ledger. Raw and derived facts with provenance. The peer-gap engine.
vera/playbooks.py	1,325	33 trigger-kind frames, each with a rich and a thin path, plus the shared specificity engines.
vera/compose.py	229	Orchestration: ledger → voice → playbook → guard → action, with rationale generation.
vera/guard.py	139	Grounding, URL, taboo, identifier, single-CTA and repetition enforcement.
vera/intent.py	210	Priority intent router, bilingual lexicons, auto-reply fingerprinting.
vera/dialog.py	460	Conversation and merchant memory; the send / wait / end decision.
vera/policy.py	190	Tick gating and ranking. Restraint lives here.
vera/lang.py	92	Voice selection and deterministic variant choice.
vera/config.py	41	Identity and policy tunables, all environment-overridable.
selftest.py	342	84 assertions over real HTTP. No API key.
bot.py · conversation_handlers.py	113	The offline entry points the main brief asks for (\$7.1, \$7.4).
make_submission.py	62	Writes <code>submission.jsonl</code> for the 30 canonical test pairs.
Dockerfile · render.yaml · fly.toml · Procfile	—	Deployment, all four pre-configured for single-worker operation.